

Project Asteria: Design and Architecture of an Autonomous Planetary Exploration Rover Control Framework and Procedural Simulation Engine

Riyan Alvi

Indian Institute of Science Education and Research Mohali
Knowledge City, Sector 81, SAS Nagar, Manauli PO, Punjab 140306, India

Abstract—This report details the design, algorithmic framework, and software implementation of Project Asteria, a modular control infrastructure for autonomous planetary exploration rovers executing within procedurally synthesized terrains. Long-range robotic space missions demand resilient software frameworks capable of managing infinite exploration fields, dynamic obstacle environments, and resource constraints without real-time human intervention. Our system introduces an unbounded chunk-based surface generator that relies on deterministic spatial hashing and multi-scale value noise to continuously simulate five distinct planetary biomes. For structural safety and navigation, the vehicle integrates a four-beam active Light Detection and Ranging (LiDAR) array with a state-driven local path recovery finite state machine (FSM). This architectural design allows the vehicle to safely execute micro-scale hazard avoidance maneuvers while preserving its macro-scale Boustrophedon lane-coverage sweep targets. System survivability is maintained through a hierarchical mission controller that dynamically arbitrates power allocations based on sensory environmental stress and energy constraints. Emulative metrics generated by our integrated engine show that this approach achieves robust path navigation, registers reliable episodic memory traces under critical system stress, and mitigates localized geometric deadlocks using a specialized kinematic stall watchdog protocol.

Index Terms—Project Asteria, Autonomous Exploration Rovers, Finite State Machines, Procedural Terrain Generation, LiDAR Raycasting, Boustrophedon Coverage Planning, Software Architecture.

I. INTRODUCTION

Autonomous robotic systems designed for planetary surfaces must navigate landscapes characterized by extreme environmental variability, unmapped geometric hazards, and communication latencies. Under these functional constraints, hardcoded path sequences or constant human telemetry loops are completely unviable. Vehicles require an integrated, layered software architecture capable of simultaneously tracking localized environmental metrics, detecting structural physical risks, mitigating immediate line-of-sight collisions, and planning systematic wide-area exploration surveys.

To address these compounding requirements, this report details *Project Asteria 1.0*, a robust control system prototype and accompanying planetary simulation framework. The primary contribution of this work is the structural decoupling of high-level cognitive mission planning, local state-based hazard avoidance, and procedural environment synthesis. This architectural isolation ensures that lower-level motor actuators

can maintain kinetic exploration and safety recovery overrides even during unexpected sensor drops or shifting macro mission priorities.

The remainder of this document is organized as follows: Section II describes the system architecture and component interactions. Section III defines the underlying mathematical formulations governing terrain mapping and vector movement profiles. Section IV details the comprehensive feature analysis of the core subsystems. Section V evaluates the system through the test harness validation and experimental metrics, while Section VI outlines the final synthesis and strategic future scope.

II. SYSTEM ARCHITECTURE AND COMPONENT DESIGN

The software architecture is structured around a decoupled, modular design that separates the rover’s cognitive logic from the physical simulation and rendering pipeline. This separation of concerns ensures that the core control algorithms—such as hazard avoidance, path planning, and mission management—operate independently of the visualization layer. The system is split into two primary operational layers: the Logical Control Layer (*asteria_core.py*) and the Simulation Environment Layer (*asteria_sim.py*).

A. Logical Control Layer (*asteria_core.py*)

This layer handles the internal state processing, environmental analysis, and behavioral execution of the rover. It contains no rendering or graphical code, allowing it to serve as a pure control loop.

- **Vehicle Kinematics and State (*AsteriaConfig*):** Manages the rover’s physical parameters (wheel radius of 20 cm, wheel base of 30 cm) and structural states (battery life, speed vectors, braking triggers). It handles local path calculations, maintains spatial coordinates (x, y) , and executes the local finite state machine for obstacle dodging.
- **Sensor Payload Management (*Environment & EnvironmentAnalyzer*):** Serves as the data ingestion point for ambient telemetry. The *Environment* class stores raw readings (temperature, luminosity, humidity, and 4-directional LiDAR distances). The *EnvironmentAnalyzer* processes these variables against baseline thresholds to flag systemic risks. It assigns

safety profiles (SAFE, RISKY, DANGER) and generates operational warnings based on extreme conditions.

- **Hierarchical Supervision (MissionController):** Dynamically shifts the rover’s macro mission profile based on internal and external hazards. It functions on an automated priority matrix, overriding open exploration tasks if power levels fall below critical thresholds ($< 20\%$) or if immediate environmental threats are flagged.
- **Area Coverage Planning (CoveragePlanner):** Provides the rover with systematic long-range directional vectors. It calculates a grid-wide path to ensure the rover efficiently scans an area instead of wandering at random.
- **Telemetry Registration (Memory & Experience):** Implements an episodic logging system. Whenever a state change occurs (such as entering a RISKY zone or executing an evasive maneuver), the system takes a snapshot of the pre-action environment, records the vehicle’s decision, logs the post-action environmental response, and commits it to a cumulative experience log.

B. Simulation Environment Layer (*asteria_sim.py*)

This layer acts as the physical sandbox and orchestration engine. It renders graphics via Pygame and simulates the environment that the control core interacts with.

- **Procedural Landscape Generator (MartianTile & PlanetaryObstacle):** Constructs an unbounded, dynamic world map. Rather than generating a massive, fixed-size map in memory, it partitions the world into 16×16 tile segments (chunks). These chunks are initialized, built out with specific tile textures (dust grains, iron veins), and scattered with geometric hazards (craters and rocks) on demand as the rover moves.
- **Active Lidar Raycaster (SimulationRover):** Simulates a 4-beam structural LiDAR system. It projects spatial tracking rays ahead, behind, and to the left and right of the rover’s current heading angle up to a maximum range of 150 cm. It detects intersections with procedural obstacles and feeds these real-time distances directly into the control layer.
- **Execution Runtime (SimulationEngine):** Drives the master operational loop at a fixed update rate (60 Hz). It coordinates data syncs between the simulation and control frameworks. Each frame, it collects sensor values, updates the navigation brain, applies kinetic modifications, handles battery drain/charge adjustments, tracks frame updates to prevent mechanical lockups, and updates the display.

III. MATHEMATICAL FORMULATIONS AND ALGORITHMIC LOGIC

A. Infinite Terrain Chunking and Deterministic Hashing

To support continuous exploration without overloading system memory, the environment generates world chunks dynamically based on the rover’s spatial coordinates. The system maintains a deterministic environment by avoiding global random seeds during terrain updates. Instead, it processes chunk coordinates (c_x, c_y) through a localized spatial hashing

algorithm to build an isolated pseudo-random number generator (RNG) for each unique sector:

$$\text{Seed}_{\text{Chunk}} = (\text{World_Seed} + c_x \cdot 73856093 + c_y \cdot 19349663) \pmod{2^{32}} \quad (1)$$

This mathematical approach ensures that if the rover leaves a sector and returns to the exact same coordinates later, the map re-generates identical tile patterns and obstacle distributions.

B. Macro-Scale Biome Blending

To prevent harsh visual and environmental transitions at chunk boundaries, the engine applies smoothstep interpolation across a low-frequency value noise grid. The noise value (V) at any given chunk position is calculated by sampling the four surrounding macro-grid nodes and blending them using a cubic polynomial curve:

$$S(t) = t^2 \cdot (3 - 2t) \quad (2)$$

$$V = \text{Interpolate}(V_{00}, V_{10}, V_{01}, V_{11}, S(f_x), S(f_y)) \quad (3)$$

Where f_x and f_y represent fractional offsets inside the current macro-cell. The resulting noise value $V \in [0, 1]$ maps directly to five unique geological biomes, each introducing distinct obstacle densities, crater-to-rock ratios, and baseline temperatures formatted in Table I.

C. Kinematic Navigation and Vector Movement

The rover’s position updates use a standard kinematic model. Given a movement step (d) and a heading angle (θ) oriented where 0° points along the positive x-axis, positional updates are calculated using standard trigonometric projections:

$$\Delta x = d \cdot \cos(\theta), \quad \Delta y = d \cdot \sin(\theta) \quad (4)$$

$$x_{t+1} = x_t + \Delta x, \quad y_{t+1} = y_t + \Delta y \quad (5)$$

D. Boustrophedon Pathing Vectors

The CoveragePlanner organizes wide-area sweeping routines by calculating a back-and-forth grid pattern across designated boundaries. It maps out long-range lanes based on set lane spacing parameters, alternating the target endpoints to create parallel rows. The desired target heading (θ_{Target}) from the rover’s current position (x, y) to the active waypoint (x_w, y_w) is continually calculated using a two-argument arctangent function:

$$\theta_{\text{Target}} = \text{atan2}(y_w - y, x_w - x) \cdot \left(\frac{180}{\pi}\right) \pmod{360} \quad (6)$$

E. Local Path Recovery Finite State Machine

When localized hazards block the sweeping path, the system uses a localized finite state machine to navigate around the obstacle and get back on track. To adjust its direction smoothly when returning to its path, the control loop isolates heading errors and prevents rapid or incorrect spinning by applying a wrapping modulus equation:

$$\Delta\theta = ((\theta_{\text{Path}} - \theta_{\text{Current}} + 180) \pmod{360}) - 180 \quad (7)$$

TABLE I
PROCEDURAL BIOME MATRIX CONFIGURATIONS

Biome Key	Base Temp.	Density	Crater Prob.	Obstacle Radii Range
polar_frost	130 K	3 / chunk	0.35	12 to 30 cm
highland_ridges	165 K	13 / chunk	0.20	20 to 55 cm
basalt_flats	195 K	4 / chunk	0.25	15 to 35 cm
crater_field	200 K	16 / chunk	0.85	25 to 60 cm
rust_plains	210 K	8 / chunk	0.50	15 to 50 cm

The vehicle uses this directional offset ($\Delta\theta$) to steer back toward its original path, turning in increments of its maximum step rate until $\Delta\theta = 0$, at which point it exits the recovery state.

IV. COMPREHENSIVE FEATURE ANALYSIS AND SUBSYSTEMS

A. Infinite Chunk-Based Procedural Map Generation

The environment operates as a continuous, unbounded space, discarding old fixed-grid map boundaries. The world space is partitioned into independent geographic coordinate blocks called chunks, measuring 16×16 tile units.

- **Dynamic Memory Management:** The engine tracks the rover's instantaneous (x, y) coordinate position, checking chunk references to maintain maps only within a set radius around the vehicle.
- **On-Demand Buffering:** Chunks entering the active boundary are initialized dynamically, while distant chunks are removed to keep memory allocations predictable.
- **Deterministic Surface Texturing:** Each chunk maintains a private, coordinate-seeded random instance. This generates static terrain features, including localized micro dust grains and structural iron veins, keeping maps identical whenever a sector is re-visited.

B. Dynamic Structural LiDAR Raycasting

The system simulates active distance-sensing hardware through a multi-directional vector raycaster.

- **Four-Beam Array Geometry:** The sensor projects four scanning vectors aligned with the vehicle's structural layout: Front (0°), Left (-90°), Right (90°), and Rear (180°) relative to the current heading.
- **Incremental Intersection Projections:** Each ray projects spatial checkpoints along its vector line, sampling for collisions with procedural obstacles up to a maximum range of 150 cm.
- **Real-Time Actuation Ingestion:** Detected obstacle distances are fed back frame-by-frame to update physical variables in the configuration core. This feeds distance values to front, rear, left, and right channels simultaneously.

C. Hierarchical Mission Controller

High-level mission routing is governed by an objective matrix that prioritizes vehicle safety over exploration parameters. The

architecture dynamically switches between four operational profiles:

- **Recharge Loop:** Triggered immediately if the battery subsystem breaches minimum safety parameters ($< 20\%$). Exploration drains are halted, and a trickle-charge current ($+0.4\%$ per frame) engages to sustain electronic structures.
- **Avoid Danger Routine:** Activated when the analyzer flags a DANGER profile, overriding active steering to run immediate evasion maneuvers.
- **Proceed Carefully Speed Limit:** Engaged under localized environment stresses or RISKY status warnings, halving maximum movement limits to reduce kinetic hazards.
- **Explore Mode:** The baseline state used when telemetry reports a SAFE status, enabling regular sweeping speeds.

D. Localized Multi-State Obstacle Evasion

When immediate clearance values drop below safety thresholds, a dedicated finite state machine overrides long-range waypoint navigation.

- **State Interruption Triggers:** Evasion routines engage when front telemetry reads less than 10 cm.
- **Directional Arbitrations:** The system reads the left and right LiDAR distance values to choose an exit path, turning toward whichever side reports greater open clearance.
- **Persistent Travel Realignment:** The system logs its heading vector at the moment of interruption to an internal register (`path_heading`). While dodging, it maneuvers around obstacles until its front sensor clears. Once unblocked, it switches to a returning loop, using a wrapping modulus formula to correct steering angles until it snaps back onto its original trajectory line.

E. Ambient Telemetry Parsing and Environmental Analysis

The payload system reads multiple ambient metrics to classify local environmental safety levels.

- **Thermal Analysis:** Processes raw environmental values into categorical profiles, mapping conditions to analogous environments like Deep Space (< 30 K), Mars (< 273 K), Venus (< 800 K), or Solar Core regions ($< 15,000,000$ K).
- **Atmospheric Stress Monitoring:** Evaluates relative humidity against terrestrial indices, identifying settings ranging from interplanetary space voids ($< 5\%$) up to thick fog ($100\% \equiv$ Sensor malfunction).

analog edge cases to evaluate system performance during state transitions:

- **Safe-State Exploration Testing:** Initializes nominal parameters—such as an ambient temperature of 25°C, clear multi-directional visibility readings of 30 cm, a baseline humidity of 40%, and a twilight luminosity reading of 3 lux. The test verifies that the system registers a `SAFE` status profile and remains in the forward `Explore` loop.
- **Low-Power Mitigation Testing:** Artificially overrides the internal cell manager to drop charge levels down to 15%. The framework verifies that the high-level `MissionController` immediately intercepts the life-cycle loop, overrides ongoing area-sweeping targets, and flags an urgent `Recharge` mandate.
- **Immediate-Impact Evasion Testing:** Simulates an immediate physical barrier by hardcoding front LiDAR distance variables down to 2 cm. The validation routine verifies that the system flags a `DANGER` alert profile, stops forward motion, and hands control over to the local finite state machine to execute avoidance maneuvers.

B. Architectural Separation Verification

The simulation engine validates the performance advantages of decoupling the software architecture. Because the low-level movement routines (`avoid_obstacles`) run continuously alongside the higher-level mission planning loops, the system prevents execution lockups. Even when mismatched string definitions are introduced to high-level goals during development, the lower-level movement core safely executes local hazard avoidance and active braking processes to protect the rover from collisions.

VI. CONCLUSION AND STRATEGIC ENHANCEMENTS

A. Project Summary

The project successfully delivers a modular software prototype for an autonomous planetary exploration rover. By strictly decoupling high-level mission routing, sensory payload parsing, local state-driven hazard mitigation, and procedural environment rendering, the system achieves autonomous survival and systematic exploration capabilities. Key milestones include implementing infinite chunk-based world updates, a deterministic map generation engine driven by coordinate-seeded spatial hashes, and a state-tracking LiDAR recovery machine that clears obstacles while preserving long-range path targets.

B. Strategic Future Enhancements

To scale this core control prototype toward actual hardware integration or high-fidelity simulation environments, development priorities include:

- **Dynamic Global A* Navigation:** Transitioning from local, sensor-blind evasive maneuvering toward real-time grid mapping, allowing the rover to plot global optimal paths through previously mapped chunks.
- **Non-Volatile Serialization Storage:** Replacing transient in-memory array tracking with a persistent database

or structured file-system schema to let the rover store extensive mission experience logs across restarts.

- **Multi-Agent Swarm Integration:** Modifying communication layers to support synchronized multi-rover fleets, enabling automated map sharing and collaborative area-sweeping operations.